Machine Learning 2

Final project presentation

Izabela Kosiec 441755

Katarzyna Pełka 432453

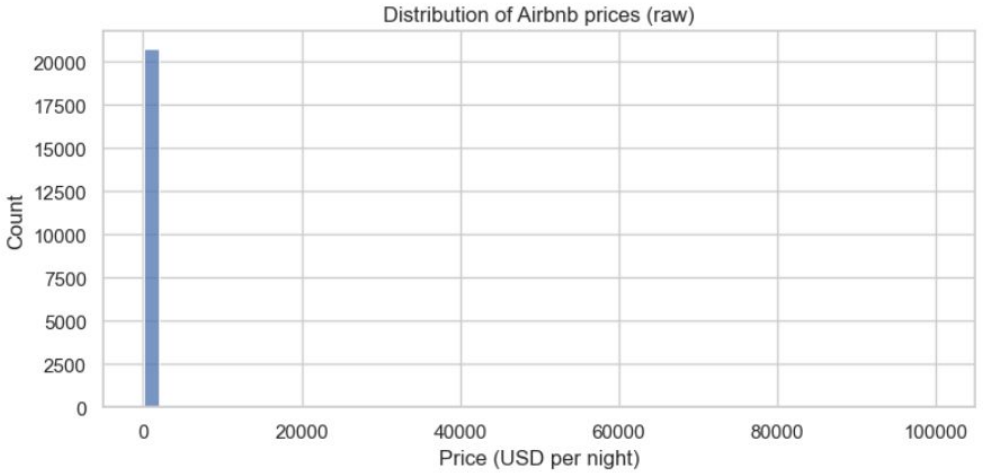
Regression - Predicting nightly prices of Airbnb listings in New York City using a regression

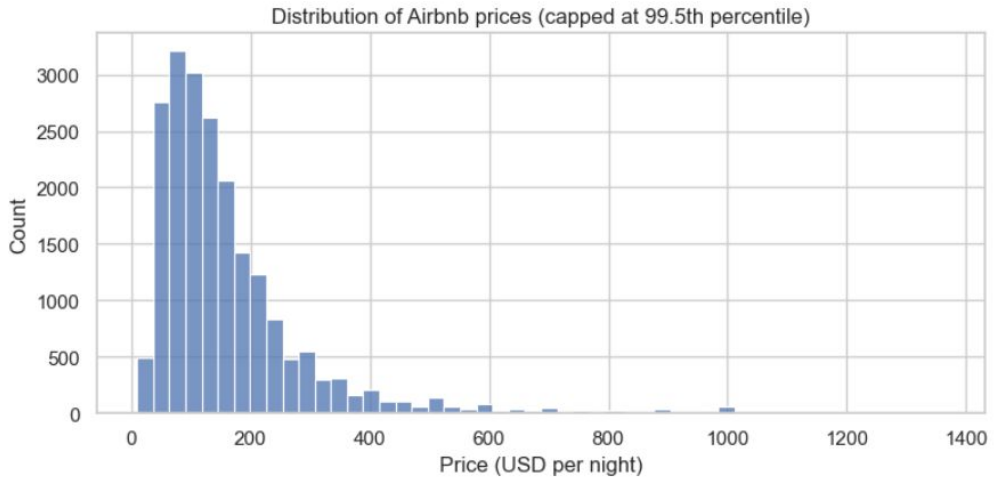
The dataset new_york_listings_2024.csv contains 20 758 observations and 22 columns describing property characteristics, host activity, location, availability, and user feedback.

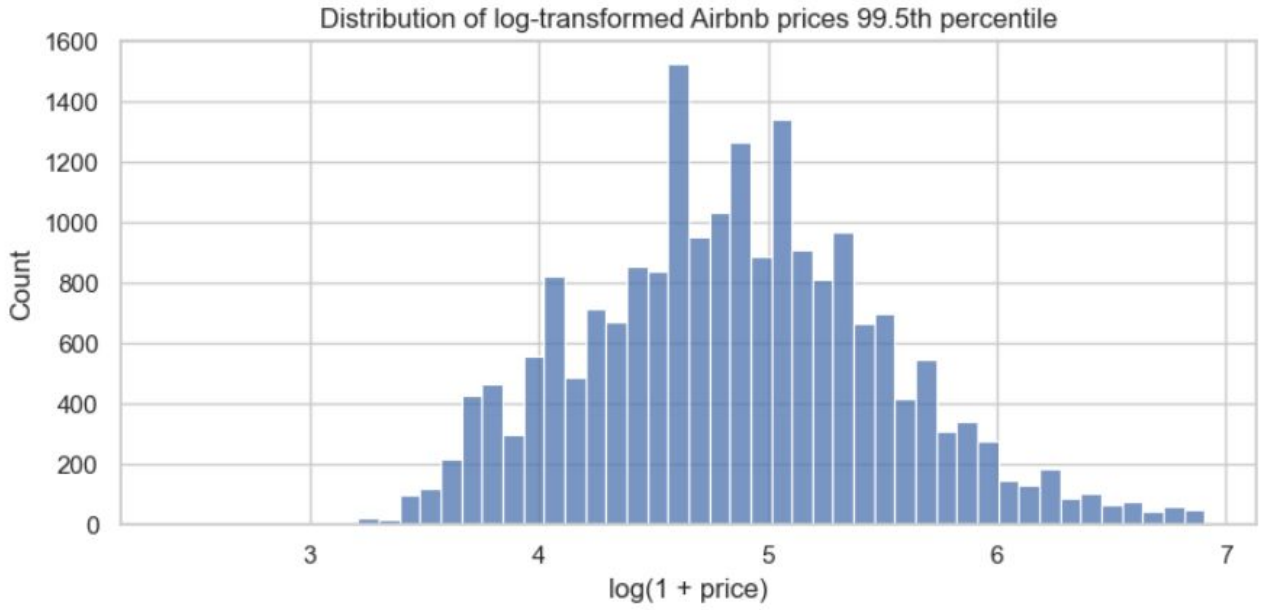
Examples of variables:

- **latitude, longitude**– geographic coordinates of the listing, capturing spatial effects within the city
- **neighbourhood_group**– high-level administrative area of New York City (categorical)
- **neighbourhood** – local neighborhood name (categorical)
- **room_type** – type of accommodation offered (Entire home/apt, Private room, Shared room)
- **bedrooms, beds, baths** – indicators of property size and capacity
- **minimum_nights** – minimum required length of stay
- **availability_365** – number of available days in a year, reflecting booking intensity
- **number_of_reviews, number_of_reviews_ltm, reviews_per_month** – measures of listing popularity and activity
- **rating** – average user rating of the listing
- **calculated_host_listings_count** – number of listings managed by the same host
- **license** – regulatory status of the listing (categorical)
- **price** - target variable

Target variable 'price'







The price distribution is highly right-skewed. Therefore, we log-transformed it to stabilize variance and reduce the influence of extreme prices. Even after log-transformation, the top 0.5% of listings exhibited extreme behavior; therefore, they were removed to improve model stability

Variables transformation

	count	mean	std	min	25%	50%	75%	max
latitude	20654.0	40.726735	0.060363	40.500314	40.684107	40.722595	40.763117	40.911147
longitude	20654.0	-73.939011	0.061429	-74.249840	-73.980590	-73.949440	-73.917305	-73.713650
minimum_nights	20654.0	28.574998	33.598605	1.000000	30.000000	30.000000	30.000000	1250.000000
number_of_reviews	20654.0	42.733078	73.646824	1.000000	4.000000	14.000000	50.000000	1865.000000
reviews_per_month	20654.0	1.261424	1.907281	0.010000	0.210000	0.650000	1.810000	75.490000
calculated_host_listings_count	20654.0	18.919289	71.079877	1.000000	1.000000	2.000000	5.000000	713.000000
availability_365	20654.0	205.768084	135.068901	0.000000	87.000000	214.000000	353.000000	365.000000
number_of_reviews_ltm	20654.0	10.895565	21.398346	0.000000	1.000000	3.000000	15.000000	1075.000000
beds	20654.0	1.713082	1.157515	1.000000	1.000000	1.000000	2.000000	18.000000
price_log	20654.0	4.860327	0.677922	2.397895	4.394449	4.836282	5.298317	7.218910
rating_numeric	20654.0	3.879895	1.839998	0.000000	4.330000	4.750000	4.910000	5.000000

Listings with:

- minimum_nights > 365 (99.9th percentile) == 365
- reviews_per_month > 30 == 30
- number reviews in last 12 months > 90 ==90
- calculated_host_listings_count > 300 ==300

- Neighbourhoods with fewer than 20 listings created category "Other"
- License variable was simplified into 3 categories: no license, Exempt and OSE
- Studio apartments were treated as containing 0 bedrooms, making variable bedrooms numerical one
- If number of baths was “not specified” it was replaced with median value, making variable baths numerical one
- rating was converted to numeric variable (rating_numeric). Anything non-numeric became NaN and was filled with 0 (meaning: no rating yet)

Final variables and splitting data:

```python
# Split into features X and targets y (raw) and y_log (log-transformed)
y = df["price"]
y_log = df["price_log"]
# Remove both target variables from feature matrix to avoid data leakage
X = df.drop(columns=["price", "price_log"])
print("\nFeature matrix shape:", X.shape)
print("Target vector (raw price) shape:", y.shape)
print("Target vector (log price) shape:", y_log.shape)

# Identify categorical and numeric columns - final ones
categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()
numeric_cols = X.select_dtypes(exclude=["object"]).columns.tolist()
print("\nCategorical columns:", categorical_cols)
print("Numeric columns:", numeric_cols)

# One-hot encoding for categorical variables
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
print("\nShape after one-hot encoding:", X_encoded.shape)
```

```python
#  Train-test split
idx_train, idx_test = train_test_split(
    df.index,
    test_size=0.2,
    random_state=RANDOM_STATE
)

X_train = X_encoded.loc[idx_train]
X_test  = X_encoded.loc[idx_test]

y_train = y.loc[idx_train]
y_test  = y.loc[idx_test]

y_train_log = y_log.loc[idx_train]
y_test_log  = y_log.loc[idx_test]

print("\nTrain shapes:", X_train.shape, y_train_log.shape)
print("Test shapes:", X_test.shape, y_test_log.shape)
```

```
Feature matrix shape: (20654, 16)
Target vector (raw price) shape: (20654,)
Target vector (log price) shape: (20654,)

Categorical columns: ['neighbourhood_group', 'neighbourhood', 'room_type', 'license']
Numeric columns: ['latitude', 'longitude', 'minimum_nights', 'number_of_reviews', 'reviews_per_month', 'calculated_host_listing
s_count', 'availability_365', 'number_of_reviews_ltm', 'bedrooms', 'beds', 'baths', 'rating_numeric']

Shape after one-hot encoding: (20654, 145)

Train shapes: (16523, 145) (16523,)
Test shapes: (4131, 145) (4131,)
```

rating, id, host_id, name, host_name, last_review were dropped, since they are identifiers or not very informative data for our model

No missing values were found after data transformations

Comparison of the best models based on RMSE value

Models taken into consideration:

- Baseline - mean predictor
- Gradient Boosting
- Random Forest
- Ensemble Learning (RF + XGB)
- XGBoost with Early stopping- winner

XGB best RMSE = 90.475141

R^2 = 0.5551

Best parameters:

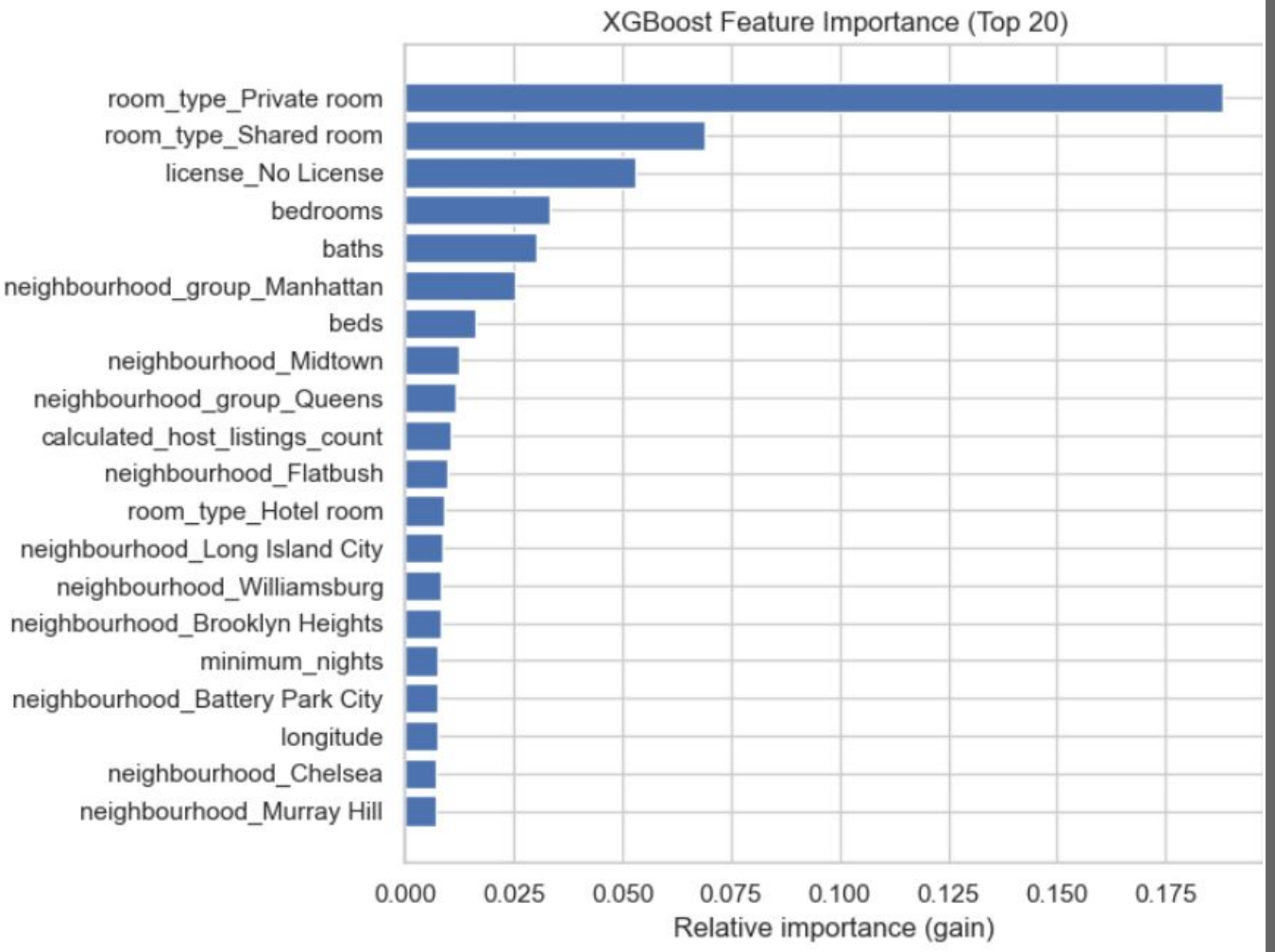
- n_estimators=909,
- max_depth=9,
- learning_rate=0.023,
- subsample=0.825,
- colsample_bytree=0.65

	Model	Test RMSE	Test R2	Notes
0	XGBoost + Early Stopping (log-price)	90.475141	0.555146	FINAL MODEL
1	Ensemble (RF + XGB)	90.993964	0.550030	No improvement over XGB
2	Random Forest (log-price)	92.667436	0.533327	Bagging + OOB validation
3	Gradient Boosting (log-price)	97.166533	0.486912	Classical boosting
4	Baseline (Mean Predictor)	139.692215	-0.060482	Naive baseline

Feature importance (final XGB model)

Top 15 most important features (by gain):

	feature	importance_gain	importance_gain_norm
141	room_type_Private room	14.888094	0.188287
142	room_type_Shared room	5.458369	0.069031
143	license_No License	4.186059	0.052940
8	bedrooms	2.632802	0.033297
10	baths	2.378443	0.030080
13	neighbourhood_group_Manhattan	1.987735	0.025139
9	beds	1.288402	0.016294
92	neighbourhood_Midtown	0.978944	0.012381
14	neighbourhood_group_Queens	0.921271	0.011651
5	calculated_host_listings_count	0.818131	0.010347
57	neighbourhood_Flatbush	0.782026	0.009890
140	room_type_Hotel room	0.694895	0.008788
86	neighbourhood_Long Island City	0.688739	0.008710
136	neighbourhood_Williamsburg	0.655285	0.008287
29	neighbourhood_Brooklyn Heights	0.650063	0.008221



Classification - Predicting one's gender based on wellbeing and lifestyle habits

The dataset Wellbeing_and_lifestyle_data_Kaggle.csv contains 15 972 responses to 24 survey questions which reflect how well people shape their lifestyle, habits and behaviours to maximize overall life satisfaction.

Questions in the survey:

- **How many fruits/vegetables do you eat every day?**
- **How much stress do you typically experience everyday?** (1-5)
- **How many new places do you visit every year?** (1-10, 10 reflected also when 10 or more)
- **How many people are you very close to?** (1-10, 10 reflected also when 10 or more)
- **How many remarkable achievements are you proud of?** (1-10, 10 reflected also when 10 or more)
- **How many times a year do you donate money or time to good causes?** (1-5, 5 reflected also when 10 or more)
- **What's your BMI range?** (binary - 2 options)
- **How many hours do you spend everyday doing what are you passionate about?** (1-10, 10 reflected also when 10 or more)
- **In typical week, how many times do you have an opportunity to think about your life (meditate)?**
- **…**
- **Work-life balance score calculated based on the answers**

Descriptive statistics

	count	mean	std	min	25%	50%	75%	max
FRUITS_VEGGIES	15972.0	2.922677	1.442694	0.0	2.0	3.0	4.0	5.0
PLACES_VISITED	15972.0	5.232970	3.311912	0.0	2.0	5.0	8.0	10.0
CORE_CIRCLE	15972.0	5.508077	2.840334	0.0	3.0	5.0	8.0	10.0
SUPPORTING_OTHERS	15972.0	5.616454	3.242021	0.0	3.0	5.0	10.0	10.0
SOCIAL_NETWORK	15972.0	6.474267	3.086672	0.0	4.0	6.0	10.0	10.0
ACHIEVEMENT	15972.0	4.000751	2.755837	0.0	2.0	3.0	6.0	10.0
DONATION	15972.0	2.715314	1.851586	0.0	1.0	3.0	5.0	5.0
BMI_RANGE	15972.0	1.410656	0.491968	1.0	1.0	1.0	2.0	2.0
TODO_COMPLETED	15972.0	5.745993	2.624097	0.0	4.0	6.0	8.0	10.0
FLOW	15972.0	3.194778	2.357518	0.0	1.0	3.0	5.0	10.0
DAILY_STEPS	15972.0	5.703606	2.891013	1.0	3.0	5.0	8.0	10.0
LIVE_VISION	15972.0	3.752129	3.230987	0.0	1.0	3.0	5.0	10.0
SLEEP_HOURS	15972.0	7.042888	1.199044	1.0	6.0	7.0	8.0	10.0
LOST_VACATION	15972.0	2.898886	3.692180	0.0	0.0	0.0	5.0	10.0
DAILY_SHOUTING	15972.0	2.930879	2.676301	0.0	1.0	2.0	4.0	10.0
SUFFICIENT_INCOME	15972.0	1.728963	0.444509	1.0	1.0	2.0	2.0	2.0
PERSONAL_AWARDS	15972.0	5.711558	3.089630	0.0	3.0	5.0	9.0	10.0
TIME_FOR_PASSION	15972.0	3.326572	2.729293	0.0	1.0	3.0	5.0	10.0
WEEKLY_MEDITATION	15972.0	6.233346	3.016571	0.0	4.0	7.0	10.0	10.0
WORK_LIFE_BALANCE_SCORE	15972.0	666.751503	45.019868	480.0	636.0	667.7	698.5	820.2

```
Column: DAILY_STRESS
DAILY_STRESS
3          4398
2          3407
4          2960
1          2478
5          2052
0           676
1/1/00        1
Name: count, dtype: int64

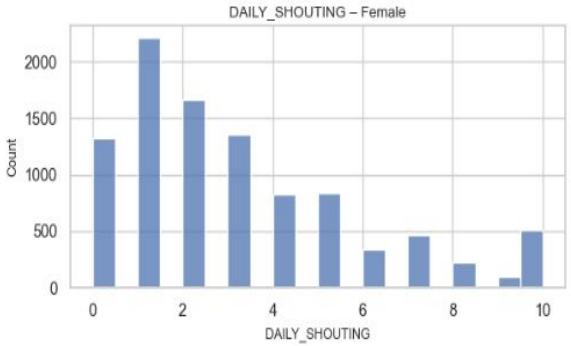
Column: AGE
AGE
21 to 35         6108
36 to 50         4655
51 or more       3390
Less than 20     1819
Name: count, dtype: int64

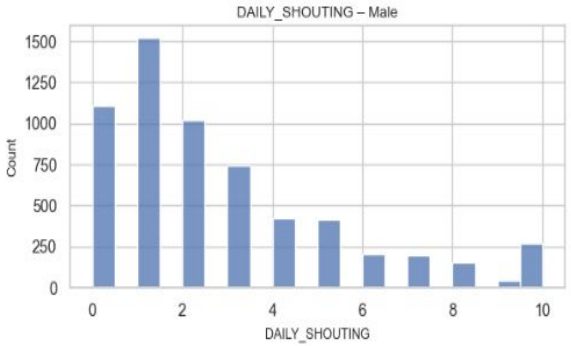
Column: GENDER
GENDER
Female    9858
Male      6114
Name: count, dtype: int64
```

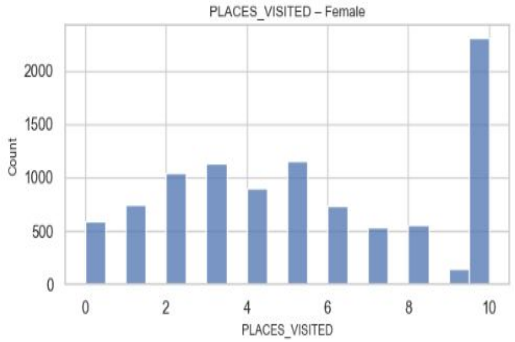
Deleted

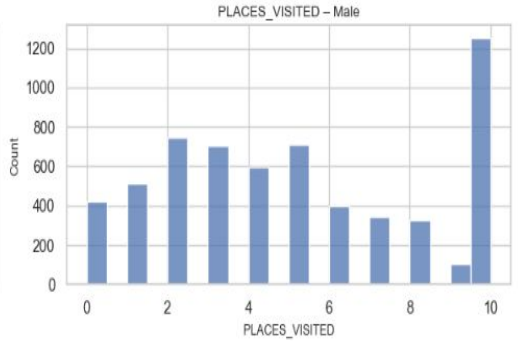
Only one possible outlier but decided to keep it as high values are there for men only so can be valuable in prediction

Distribution of the variables for men vs for women



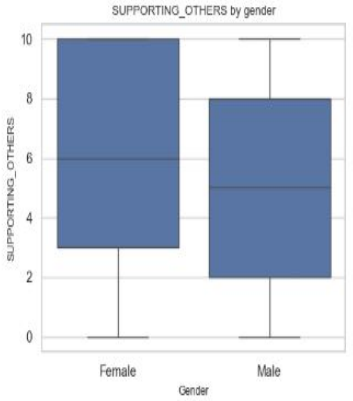


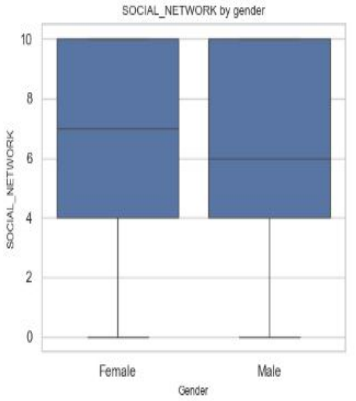


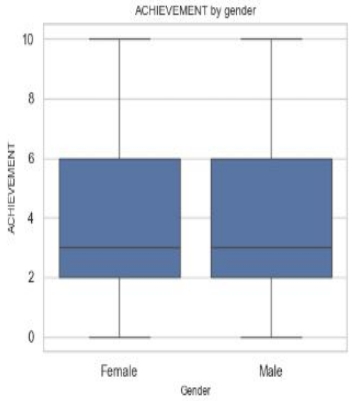
















Models - first attempt (with all variables)

- **Logistic Regression**
- **Random Forest**
- **Gradient Boostin**
- **XGBoost**

Handling class imbalance:

Logistic regression trained with balanced class weights (which significantly improves recall for minority class - men)

For XGBoost scale_pos_weight parameter was used, resulting in more balanced precision–recall trade-off and the highest overall F1-score

	model	train_accuracy	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
0	Logistic Regression (tuned)	0.628943	0.620970	0.504011	0.616517	0.554616	0.672390
3	XGBoost (tuned)	0.809267	0.663537	0.584475	0.418643	0.487851	0.686862
1	Random Forest (tuned)	0.999687	0.686385	0.663220	0.367130	0.472632	0.709207
2	Gradient Boosting (tuned)	0.694138	0.661972	0.610510	0.322976	0.422460	0.678829

Overfitting!

Feature Selection and final results

```
Correlation of numeric features with gender (Male=1, Female=0):
WEEKLY_MEDITATION          0.087545
DAILY_STEPS                0.037650
LIVE_VISION                0.030817
LOST_VACATION              0.025780
TIME_FOR_PASSION           0.016590
SUFFICIENT_INCOME          0.010419
ACHIEVEMENT                0.000598
BMI_RANGE                 -0.010766
PERSONAL_AWARDS           -0.013848
FLOW                      -0.017370
WORK_LIFE_BALANCE_SCORE   -0.039911
SOCIAL_NETWORK            -0.041020
PLACES_VISITED            -0.042662
SLEEP_HOURS               -0.058400
DAILY_SHOUTING            -0.064605
TODO_COMPLETED            -0.081400
CORE_CIRCLE               -0.096481
FRUITS_VEGGIES            -0.110211
DONATION                  -0.118754
SUPPORTING_OTHERS         -0.131476
dtype: float64
```

| Correlation | < 0.02 dropped

	model	train_accuracy	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
3	XGBoost (reduced, tuned, class-weighted)	0.673554	0.630986	0.515320	0.605070	0.556600	0.681830
0	Logistic Regression (reduced, tuned)	0.629725	0.612520	0.495010	0.608340	0.545855	0.668325
2	Gradient Boosting (reduced, tuned)	0.750724	0.666041	0.599237	0.385119	0.468890	0.680749
1	Random Forest (reduced, tuned)	0.851295	0.676995	0.670232	0.307441	0.421525	0.691213

- Overfitting reduced
- Recall for XGBoost improved significantly, also making it the best model in terms of F1 score
- Results still are not super good (Gradient Boosting and Random Forest even worse than a coin toss), but given the characteristic of the data this was expected.



Thanks for listening!